

Spring Boot 3.2.8 개발 심화

inky4832@daum.net

6. Spring Security 활용한 보안과 인증

- Spring Security 개요
- 인증/권한
- JWT

6. 보안과 인증

학습목표

- ◆ Spring Security 개요 및 특징
- ◆ Spring Security 보안 기능
- ◆ CSRF 및 CORS 개요
- ◆ Spring Security 주요 API
- ◆ Spring Security 아키텍처
- ◆ Spring Security 인증방식
- ◆ Token 개요
- ◆ JWT(Json Web Token) 개요
- ◆ JWT 동작 흐름
- ◆ Refresh Token

6. 보안과 인증

Spring Security

■ 개념

- Spring 기반의 어플리케이션 보안 표준 프레임워크
- 인증(Authentication)과 인가(Authorization)를 일관된 정책으로 처리할 수 있도록 설계
- 개발자가 보안 처리에 직접 신경 쓰는 작업을 최소화

■ 의존성

```
dependencies {  
  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
  
    implementation 'org.springframework.boot:spring-boot-starter-security'  
  
}
```

6. 보안과 인증

Spring Security 처리 영역

구분	설명	예시
인증 (Authentication)	사용자 확인	로그인, 토큰 검증
인가 (Authorization)	권한 확인	관리자만 삭제 가능
보호 (Security)	위협 방지	CSRF/XSS/CORS 방지
감사 (Logging/Auditing)	기록 관리	접속 기록, 실패 시도 로그

6. 보안과 인증

Spring Security 특징

구분	설명
강력한 보안 & 표준지원	업계 보안 표준(인증,인가,암호화 등) 기반 / 최신 공격 방어 기능 내장
유연한 확장성	세션/토큰(JWT)/ OAuth2 연동, 다양한 인증 방식 자유롭게 적용 가능
생태계 통합	Spring Framework 및 생태계와 완벽한 호환 (Spring MVC, JPA 등)
간편한 구성	보안 필터 및 정책 기본 제공 (설정만으로 빠른 적용 가능)
필터기반 아키텍처	요청이 Controller에 도달하기 전 필터 체인에서 보안 검증 처리 Security Filter Chain이라는 보안 필터와 파이프라인을 거치며 인증/인가/예외 처리 등을 수행

6. 보안과 인증

Spring Security 보안 기능

■ 개념

- 가장 핵심 기능인 인증과 권한부여 외에도 보안과 관련된 몇 가지 기능들을 지원함

■ 기능

보안 기능	설명
CORS (Cross-Origin Resource Sharing)	기본적으로 서로 다른 domain간에는 요청을 해서 데이터 공유 불가 이것을 가능하도록 처리해서 origin 간에 리소스 공유 설정
CSRF (Cross Site Request Forgery)	사이트간 요청 위조. Spring Security를 추가하면 CSRF 보호가 자동 지원
로그인 페이지 / 로그아웃 페이지	Spring Security를 추가하면 기본적으로 로그인 페이지와 로그아웃 페이지를 자동 지원
Exception Translation Filter	인증과 권한부여와 관련하여 예외가 발생할 때마다 세부적인 예외정보를 그대로 사용자에게 보내지 않고, 적절한 HTTP 응답으로 변환해서 처리

6. 보안과 인증

CSRF (Cross-Site Request Forgery: 사이트간 요청 위조)

■ 개념

- 사용자가 특정 웹사이트에 로그인하면 사용자 세션(Session)이 생성되고, 이 사용자 세션은 브라우저 쿠키를 사용해 식별됨
- 사용자가 현재 웹사이트에서 로그 아웃하지 않고 악성 웹사이트로 이동하면, 악성 웹사이트에서는 사용자의 이전 인증정보인 쿠키를 사용할 수 있게 됨
- 이러한 취약성을 CSRF (사이트 간 요청 위조)라고 부름

■ 대응방법

보안 기능	설명
동기화 토큰 패턴	요청할 때마다 새로운 토큰을 생성
Same Site 쿠키 이용	Set-Cookie: Same Site : Strict 설정 값을 Strict로 설정해서 Same Site 쿠키를 사용하게 되면 해당 사이트로만 전송됨
REST API에서 CSRF 비활성화	상태를 저장하지 않는 REST API 경우에는 CSRF가 필요하지 않음.

6. 보안과 인증

CSRF 비활성화 예시 (REST 기반)

@Bean

```
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {  
    http  
        .csrf(csrf -> csrf.disable())  
        .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))  
        .authorizeHttpRequests(auth -> auth  
            .requestMatchers("/auth/**").permitAll()  
            .anyRequest().authenticated()  
        )  
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);  
  
    return http.build();  
}
```

6. 보안과 인증

CORS (Cross Origin Resource Sharing: 사이트 간 자원 공유)

■ 개념

- 기본적으로 서로 다른 domain간에는 서로 간에 요청을 허용하지 않음
- 그런데 domain간 허용되는 요청을 설정할 수 있는 규칙이 CORS임

■ 설정방법

방법	설명
글로벌 설정	모든 Controller, 리소스에 적용 Cors Configuration Source 빈 생성 (Spring Security 6+ 권장)
로컬 설정	특정 요청 메서드나 특정 Controller에 적용 @CrossOrigin 어노테이션 지정 ex. @CrossOrigin @CrossOrigin(origins="http://example.com")

6. 보안과 인증

CORS 글로벌 설정 예시

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(List.of("http://localhost:5173", "http://localhost:3000"));
    cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"));
    cfg.setAllowedHeaders(List.of("*"));
    cfg.setAllowCredentials(true);
    UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
    src.registerCorsConfiguration("/**", cfg);
    return src;
}
```

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .cors(cors -> cors.configurationSource(corsConfigurationSource())) // CORS 활성화
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
        );

    return http.build();
}
```

6. 보안과 인증

Spring Security 주요 API

용어	설명
Authentication (인증 정보)	누구인지를 담은 토큰 객체 ex. UsernamePasswordAuthenticationToken
Principal (주체)	인증된 사용자 객체 (일반적으로 UserDetails)
Granted Authority (권한)	사용자가 가진 권한 문자열 ex. ROLE_ADMIN
Security Context / Security Context Holder	현재 사용자의 Authentication 정보 저장 여기에는 Principal과 Authorities 가 포함
Authentication Manager / Provider Manager Authentication Provider	인증 시도 처리 ex. 비밀번호 검증 등 인증 성공 시 인증 정보가 저장된 Authentication 객체를 반환
UserDetails Service	실제 인증 작업 담당. (load User By Username () 이용) DB 또는 메모리에 있는 실제 저장된 데이터를 기반으로 인증여부 결정
Password Encoder	비밀번호 해시 처리 ex. BCryptPasswordEncoder
Filter	HTTP 요청 단계에서 작동하는 보안 요소 JWT라면 커스텀 Once Per Request Filter를 작성해서 토큰 검증

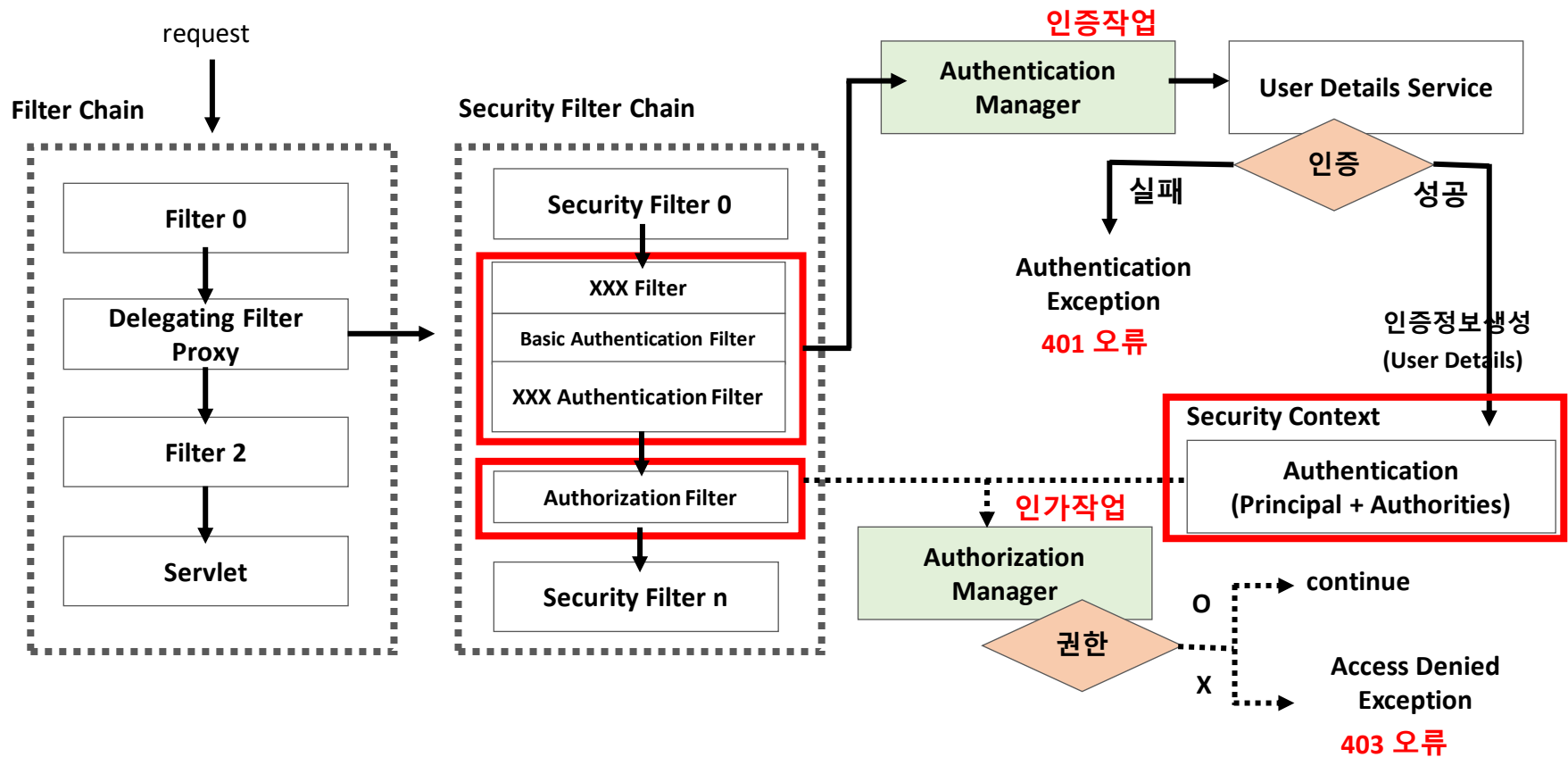
6. 보안과 인증

Spring Security 기본 3가지 구조

- Authentication Manager
 - 인증 처리 인터페이스
 - 기본 구현체는 ProviderManager이며 어떤 식으로 인정할 것인지에 따라 하위 클래스 작성
 - 인증 성공 시 인증 정보를 담고 있는 Authentication 객체를 반환
 - 인증 실패 시 AuthenticationException (401 오류)이 발생
- SecurityContext
 - 현재 사용자에게 대한 Authentication 정보가 저장
 - 여기에는 Principal과 Authorities가 포함
- Authorization Manager
 - 인증된 사용자가 접근 권한이 있는지 확인

6. 보안과 인증

Spring Security 아키텍처



6. 보안과 인증

Filter와 DelegatingFilterProxy

■ 개념

- Servlet 기반의 Application에서 실행되는 Spring Security는 filter 기반으로 동작
- 이 filter는 목적에 따라서 여러가지가 제공
- 여러 개의 필터가 연결해서 동작되는 것을 Filter Chain이라고 부름
- DelegatingFilterProxy는 Servlet과 Spring 사이의 연결 역할 담당

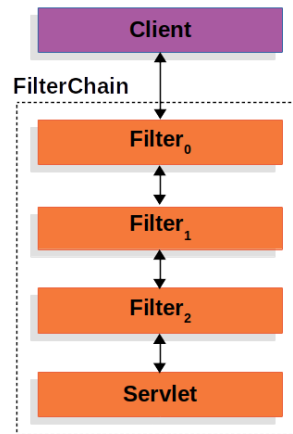


Figure 1. FilterChain

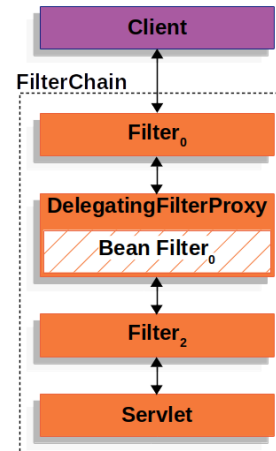


Figure 2. DelegatingFilterProxy

6. 보안과 인증

FilterChainProxy와 SecurityFilterChain

■ 개념

- FilterChainProxy는 빈이면서 Filter이고 SecurityFilterChain을 통해서 많은 Filter 처리를 위임
- 즉, Filter로 등록되는 것은 DelegatingFilterProxy인데 실제 동작하는 것은 FilterChainProxy 임
- FilterChainProxy는 모든 보안 필터들을 관리하고 조율하는 역할 담당

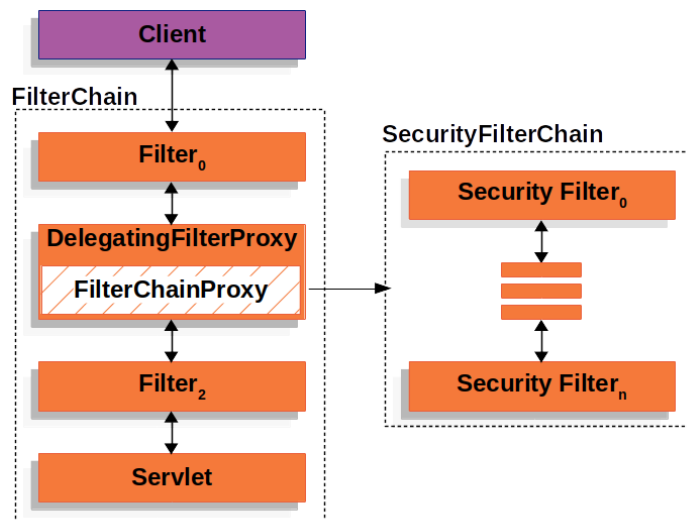


Figure 4. SecurityFilterChain

6. 보안과 인증

Security Filters

■ 개념

- Security Filters는 SecurityFilterChain API를 통해서 FilterChainProxy에 삽입됨
- 사용자 인증(authentication), 권한 부여(authorization), 공격으로부터의 보호 등 기능 수행
- 이러한 filter들은 특정 순서로 실행되어 적시에 호출되도록 보장

■ 주요 Filter 목록

종류	설명
Csrf Filter	CSRF (Cross-Site Request Forgery) 공격 방지
Logout Filter	로그아웃 처리 및 관련 핸들러 실행
Username Password Authentication Filter	사용자명/비밀번호기반 인증 처리
Anonymous Authentication Filter	인증되지 않은 사용자에게 익명 인증 객체 제공
Exception Translation Filter	보안 관련 예외(인증/인가 실패)처리
Authorization Filter	최종적인 요청 권한 검증 및 접근 제어

6. 보안과 인증

Security Filters 설정

■ 설정 방법

- Security Filters는 대부분 HttpSecurity 인스턴스를 사용해서 설정

ex.

@Configuration

@EnableWebSecurity

public class SecurityConfig {

 @Bean

 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

 http

 .csrf(Customizer.withDefaults())

 .formLogin(Customizer.withDefaults())

 .authorizeHttpRequests(authorize -> authorize

 .anyRequest().authenticated()

);

 return http.build();

 }

}

 # CsrfFilter 설정

 # UsernamePasswordAuthenticationFilter 설정

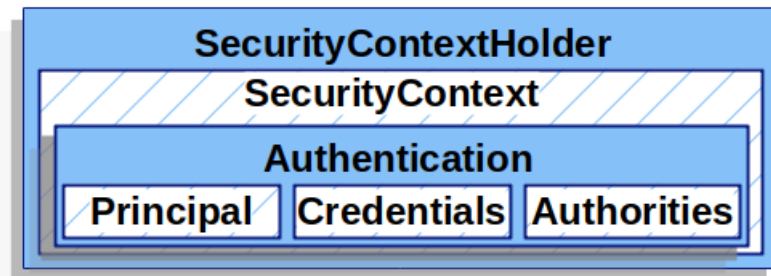
 # AuthorizationFilter 설정

6. 보안과 인증

SecurityContextHolder

■ 개념

- 인증된 사용자의 정보를 저장하는 저장소
- 사용자 인증 정보는 Security Context가 가짐



6. 보안과 인증

SecurityContext

■ 개념

- 현재 인증된 사용자의 Authentication 객체를 가짐
- Authentication은 Principal(사용자 식별 정보), Credentials(비밀번호), Authorities(권한) 포함

구성	설명
Principal	사용자를 식별하는 정보 username/password를 통해서 인증을 처리할 때 User Details 타입 객체를 사용
Credentials	password에 해당하며 일단 인증이 완료되면 보안을 위해 삭제하는 경우 많음
Authorities	Granted Authority 인스턴스로 ROLE_ADMIN, ROLE_USER 등 권한을 의미 User Details Service에 의해서 로딩됨

6. 보안과 인증

AuthenticationManager

■ 개념

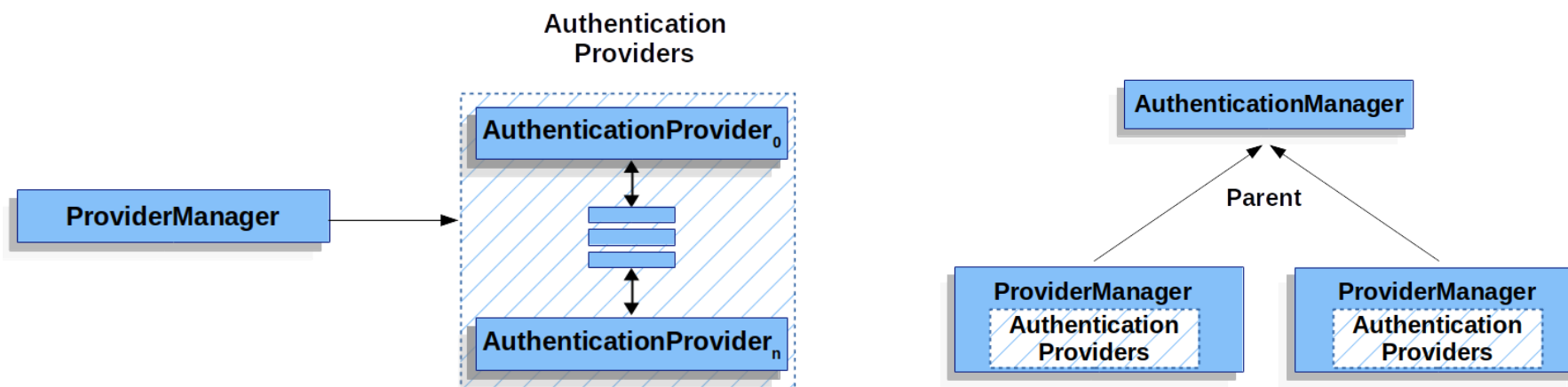
- 인증 작업을 수행하는 방법을 정의한 인터페이스
- 사용자의 인증 정보를 받아서 검증하는 역할을 수행
- 반환된 인증은 호출한 filter에 의해서 SecurityContextHolder에 저장됨
- 가장 일반적인 구현은 ProviderManger 임

6. 보안과 인증

ProviderManager

■ 개념

- 실제 인증작업을 수행하는 객체
- 여러 AuthenticationProvider 인스턴스들에게 인증 처리를 위임
- AuthenticationProvider는 순차적으로 작업하고 인증이 실패하면 예외 발생
ex. JwtAuthenticationProvider, DaoAuthenticationProvider



6. 보안과 인증

AuthorizationManager

■ 개념

- 메서드 호출이나 웹 요청과 같은 보안 객체에 대한 요청을 가로챈
- 이후 호출을 진행할 수 있는지 여부에 대한 호출 전 결정 및 주어진 값을 반환할 수 있는지에 대한 호출 후 결정 작업을 담당

■ Authorization (권한)

- 사용자가 인증되면 Authorization 객체가 생성됨
- 이 객체는 사용자가 가진 권한을 나타내는 GrantedAuthority 목록을 포함
- GrantedAuthority는 사용자(Principal)가 특정 작업을 수행할 수 있는지를 반환하는 역할

ex. "ROLE_ADMIN", "ROLE_USER"

6. 보안과 인증

Spring Security 인증 방식

구분	Form	Basic	JWT
인증 수단	세션 쿠키 (전통적)	Header (Base64)	Header (Bearer Token)
서버 상태 유지	Session 사용	Session 미사용	Session 미사용
사용 용도	Web UI	단순 API	REST API / MSA
만료 처리	세션 만료	없음	명시적 만료 포함
보안 수준	중	낮음	높음

6. 보안과 인증

Token

■ 개념

- 서버가 사용자를 식별하기 위한 디지털 신분증
즉, 사용자가 로그인에 성공했다는 증거를 담고 있는 데이터로서 매 요청마다 서버에 제출

■ 특징

특징	설명
자체 정보 포함	토큰 안에 사용자 정보 / 권한 포함
StateLESS (무상태)	서버는 사용자 상태 보관 안함
HTTP 요청에 첨부	일반적으로 Authorization: Bearer <TOKEN> 헤더로 전송
유효 시간	만료 시간 설정
서버 간 공유	여러 서버가 같은 토큰을 신뢰할 수 있어 확장 용이

6. 보안과 인증

Session 인증 vs Token 인증

특징	세션(Session) 인증	토큰(Token/JWT) 인증
상태관리	Stateful (서버가 로그인 상태를 기억)	Stateless (서버는 저장 안하고 클라이언트가 토큰을 매번 가지고 다님)
저장 위치	서버 메모리/세션 저장소 ex. Redis	클라이언트 (LocalStorage/SessionStorage/쿠키)
인증 전달 방식	Session ID가 쿠키에 자동 첨부	헤더에 Authorization: Bearer <TOKEN> 이용
만료/갱신	서버에서 직접 관리	토큰 내부 exp 기반, 갱신은 Refresh Token 필요
로그아웃 처리	서버 Session 강제 만료로 즉시 적용	토큰은 만료 전까지 유효
서버 부담	세션 저장으로 메모리 부담 증가	인증 정보를 서버에 저장하지 않아 부담 적음
CORS 처리	일반적으로 이슈 적음	CORS 헤더 설정 필수
구현 난이도	쉬움	상대적으로 복잡
대표 적용 사례	전통 Web App (JSP, Thymeleaf)	SPA, Mobile, Microservices

6. 보안과 인증

JWT (Json Web Token)

■ 개념

- RTC7519에 선언된 개방형 업계 표준
- 두 당사자 간에 클레임(데이터)를 안전하게 표현하기 위한 방법
- JSON 기반이기 때문에 대부분의 언어/플랫폼에 독립적

■ 의존성

`implementation 'io.jsonwebtoken:jjwt-api:0.11.5'`

`runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.11.5'`

`runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.11.5'`

6. 보안과 인증

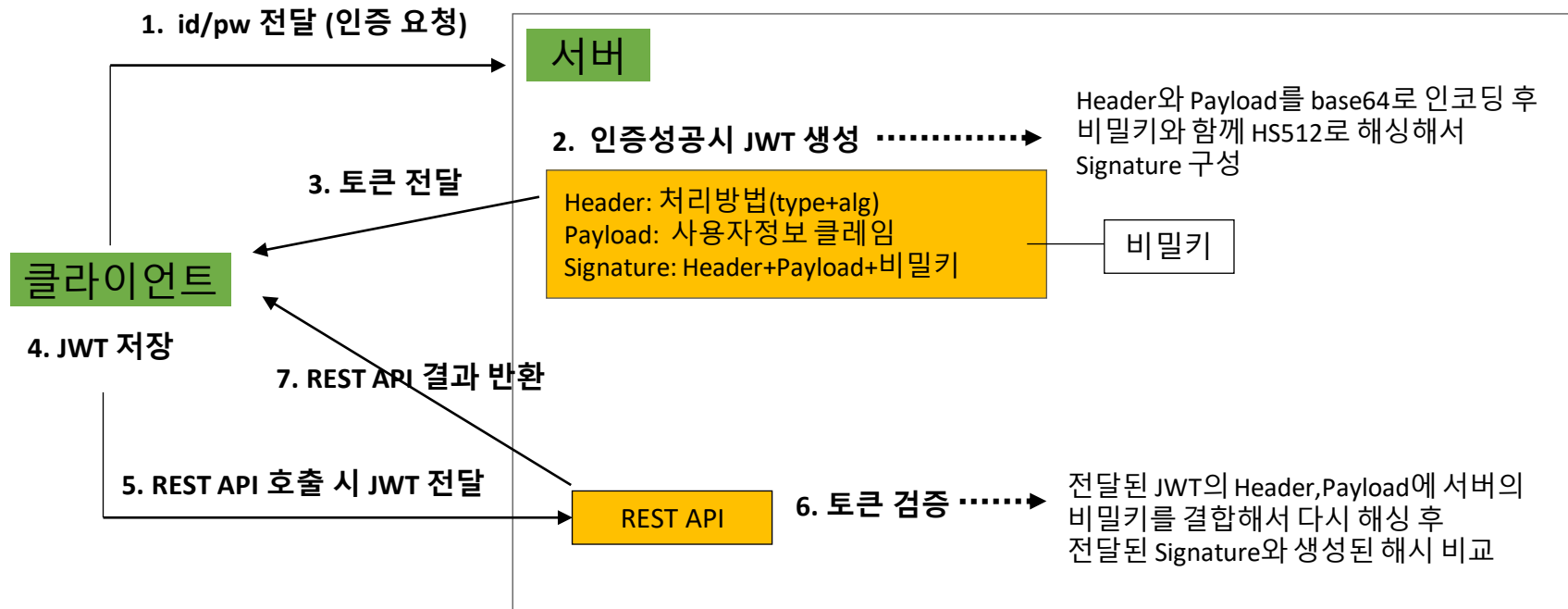
구성 요소

구성	설명	예시
Header	토큰의 유형(type)과 사용중인 알고리즘(alg)으로 구성 Base64로 인코딩 되어 토큰의 첫 번째 부분을 구성	{ "alg": "HS256", "typ": "JWT" }
Payload	사용자 정보인 클레임 (이름/권한/유효시간 등) 포함	{ "sub": "user1", "role": "ADMIN" }
Signature	위조 방지 서명 포함	HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)



6. 보안과 인증

JWT 기본 동작 흐름



6. 보안과 인증

Refresh Token

■ 개념

- JWT는 세션을 사용할 수 없는 REST 환경에서 인증 정보를 유지하기 위해 사용
- 토큰은 세션처럼 유효기간이 자체적으로 연장이 안되고 생성되는 시점에 이미 정해짐 따라서 장기간 사용 시 번거로움이 발생할 수 있음
- 너무 길면 보안에 취약하고 너무 짧으면 매번 로그인 필요
- 이러한 문제점을 해결하기 위해서 refresh token 사용

■ 역할

역할	설명
Access Token	짧은 기간 동안만 API 접근 허용
Refresh Token	장기적으로 사용자를 식별해 Access Token 재발급
핵심 아이디어	사용자는 로그인 한 번만 하면 Access Token이 만료되어도 Refresh Token으로 자동 연장 가능

6. 보안과 인증

Access Token vs Refresh Token

구분	Access Token	Refresh Token
주요 목적	인증된 사용자가 보호된 API에 접근하도록 권한을 증명	만료된 Access Token을 사용자 재인증 없이 다시 발급
수명	짧다. ex. 5분, 30분	길다 ex. 7일, 14일, 30일
사용 주체	클라이언트가 매 요청마다 서버에 제시	클라이언트가 필요할 때(access 만료 시)만 서버에 제시
전송 위치	헤더: Authorization: Bearer <access_token>	보통 HttpOnly Secure Cookie 또는 서버 DB
서버 저장 여부	Stateless, 서버가 상태를 저장하지 않음	실무에서는 서버(DB/Redis)에 저장해서 무효화 및 로테이션 가능하도록 지원
누출 시 위험도	비교적 낮음 (수명이 짧음)	높음, 반드시 안전한 저장소에 저장 필요
만료 시 처리	401 응답, 클라이언트가 /auth/refresh 로 요청해서 새로운 Access Token 재발급	refresh 자체가 만료되면 로그인 화면으로 강제 재인증
발급 시점	로그인 성공 시 무조건 발급	로그인 성공 시 동시에 발급하거나 별도 토큰 갱신 시점에 발급

6. 보안과 인증

실습 프로젝트 구조 (JWT)

```
src/main/java/
  com/example/demo
    DemoApplication.java                # [SpringBootApplication]
    config/SecurityConfig.java          # Spring Security 설정 클래스
      RedisConfig.java                  # Redis 설정, refreshToken 저장
    controller/AuthController.java, MemberController.java  # @RestController,
    dto/LoginDTO.java, MemberDTO.java, TokenRefreshDTO.java  # DTO
    entity/Member.java, Role.java       # Entity, @Entity
    exception/GlobalExceptionHandler.java  # 예외처리
      RestAccessDeniedHandler.java, RestAuthenticationEntryPoint.java
    repository/MemberRepository.java     # JpaRepository<Member, Long>
    security/CustomeUserDetails.java     # UserDetails 커스텀 클래스
      JwtFilter.java                     # 매 요청마다 동작되는 JWT 필터
      JwtUtil.java                       # JWT 생성/파싱 (username 추출)
      MyUserDetailsService.java          # 사용자 정보 로드(load)

src/main/resources
  application.yml                       # Spring JPA 연동 설정
```

6. 보안과 인증

실습(JWT)

build.gradle

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'  
    implementation 'org.springframework.boot:spring-boot-starter-data-redis'  
    implementation 'org.springframework.boot:spring-boot-starter-security'  
    runtimeOnly 'com.mysql:mysql-connector-j'  
    compileOnly 'org.projectlombok:lombok:1.18.34'  
    annotationProcessor 'org.projectlombok:lombok:1.18.34'  
  
    // JWT  
    implementation 'io.jsonwebtoken:jjwt-api:0.11.5'  
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.11.5'  
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.11.5'  
  
    developmentOnly 'org.springframework.boot:spring-boot-devtools'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
}
```

6. 보안과 인증

실습(JWT)

application.yml

server:

port: 8090

spring:

mvc:

pathmatch:

matching-strategy: ant_path_matcher

datasource:

url: jdbc:mysql://localhost:3306/testdb

username: root

password: 1234

driver-class-name: com.mysql.cj.jdbc.Driver

hikari:

pool-name: app-hikari

maximum-pool-size: 20

minimum-idle: 5

idle-timeout: 600000 # 10분

max-lifetime: 1800000 # 30분

connection-timeout: 30000 # 30초

jpa:

open-in-view: false

hibernate:

ddl-auto: create # 운영 권장: 스키마 자동 변경 금지

properties:

hibernate:

format_sql: true # SQL 포매팅

data:

redis:

host: localhost

port: 6379

logging:

level:

root: info

org.springframework.web: debug

org.hibernate.SQL: trace # 실행되는 SQL

org.hibernate.orm.jdbc.bind: trace # 바인딩 파라미터

org.hibernate.orm.jdbc.extract: trace # 결과 매핑/추출

org.springframework.security: DEBUG

jwt:

secretKey:

rgRmR6aE+TpiGTgthuxAABo7z5jXfysZzhkO57zXf9U9xM+YD0CIHtY2M+5o9hE7Kf
EhuyrA/RfBAI2+QeyD5w== # 암호화 + Base64 인코딩처리

secretKeyRt:

GSXgsGbxTHQPE3NXTTah6NzQmAcFKlhE+VdsTimUmCMQ9ZZSdOcPorxASPAHBd
DM7QBUBjMGdKBn1vW9iZAlbA==

6. 보안과 인증

실습(JWT)

```
// SecurityConfig.java
package com.example.demo.config;

@Configuration
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtFilter jwtFilter;
    private final RestAuthenticationEntryPoint restAuthenticationEntryPoint;
    private final RestAccessDeniedHandler restAccessDeniedHandler;
    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .cors(c -> c.configurationSource(corsConfigurationSource()))
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(restAuthenticationEntryPoint)
                .accessDeniedHandler(restAccessDeniedHandler))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

6. 보안과 인증

실습(JWT)

```
//비밀번호 암호화
@Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
//AuthenticationManager 설정
@Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
//CORS 설정
@Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration cfg = new CorsConfiguration();
        cfg.setAllowedOrigins(List.of("http://localhost:5173", "http://localhost:3000"));
        cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"));
        cfg.setAllowedHeaders(List.of("*"));
        cfg.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
        src.registerCorsConfiguration("/**", cfg);
        return src;
    }
}
```

6. 보안과 인증

실습(JWT)

```
// RedisConfig.java
package com.example.demo.config;

@Configuration
public class RedisConfig {

    @Value("${spring.data.redis.host}")
    private String host;

    @Value("${spring.data.redis.port}")
    private int port;

    @Bean("rtdb")
    public RedisConnectionFactory redisConnectionFactory(){

        RedisStandaloneConfiguration configuration = new
RedisStandaloneConfiguration();
        configuration.setHostName(host);
        configuration.setPort(port);
        configuration.setDatabase(0);
        return new LettuceConnectionFactory(configuration);
    }
}
```

```
@Bean
public RedisTemplate<String, String> redisTemplate(
    @Qualifier("rtdb") RedisConnectionFactory redisConnectionFactory){

    RedisTemplate<String, String> redisTemplate = new RedisTemplate<>();
    redisTemplate.setKeySerializer(new StringRedisSerializer());
    redisTemplate.setValueSerializer(new StringRedisSerializer());
    redisTemplate.setConnectionFactory(redisConnectionFactory);
    return redisTemplate;
}

}
```

6. 보안과 인증

실습(JWT)

```
// CustomUserDetails.java
package com.example.demo.security;

import java.util.Collection;
import java.util.List;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import lombok.Getter;

@Getter
public class CustomUserDetails implements UserDetails {

    // 필수 구현 항목
    private String username; //email 사용
    private String password;
    private Collection<? extends GrantedAuthority> authorities; //권한목록
    private String role;
```

6. 보안과 인증

실습(JWT)

```
public CustomUserDetails(String username, String password, String role) {
    this.username = username;
    this.password = password;
    this.authorities = List.of(
        new SimpleGrantedAuthority("ROLE_" + role)
    );
}

@Override
public boolean isAccountNonExpired() {
    return true; // 계정 만료 여부
}

@Override
public boolean isAccountNonLocked() {
    return true; // 계정 잠김 여부
}

@Override
public boolean isCredentialsNonExpired() {
    return true; // 비밀번호 만료 여부
}

@Override
public boolean isEnabled() {
    return true; // 계정 활성화 여부
}
}
```

6. 보안과 인증

실습(JWT)

```
// JwtUtil.java
package com.example.demo.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import java.util.*;

@Component
@Getter
public class JwtUtil {

    @Value("${jwt.secretKey}")
    private String secretKey;

    @Value("${jwt.secretKeyRt}")
    private String secretKeyRt;

    private SecretKey ACCESS_KEY;
    private SecretKey REFRESH_KEY;
```

```
@PostConstruct
public void init() {
    byte[] accessKeyBytes = Base64.getDecoder().decode(secretKey);
    byte[] refreshKeyBytes = Base64.getDecoder().decode(secretKeyRt);

    ACCESS_KEY = Keys.hmacShaKeyFor(accessKeyBytes);
    REFRESH_KEY = Keys.hmacShaKeyFor(refreshKeyBytes);
}
```

6. 보안과 인증

실습(JWT)

```
public String createToken(String email, String role) {  
  
    // (1) Claims 생성 + subject 지정 (  
    Claims claims = Jwts.claims().setSubject(email);  
  
    // (2) 사용자 권한(ROLE) 저장 → downstream에서 인가에 활용 가능  
    claims.put("role", role);  
  
    // (권장) Access/Refresh 구분을 위해 type 클레임을 넣는 것이 안전  
    claims.put("type", "access");  
  
    Date now = new Date();  
  
    // 운영 예시: 15분  
    // Date exp = new Date(now.getTime() + 1000L * 60 * 15);  
  
    // 실습용: 30초  
    Date exp = new Date(now.getTime() + 1000L * 30);  
  
    return Jwts.builder()  
        .setClaims(claims)  
        .setIssuedAt(now)  
        .setExpiration(exp)  
        .signWith(ACCESS_KEY, SignatureAlgorithm.HS512)  
        .compact();  
}
```

```
public String createRefreshToken(String email, String role) {  
  
    Claims claims = Jwts.claims().setSubject(email);  
    claims.put("role", role);  
  
    claims.put("type", "refresh");  
  
    Date now = new Date();  
    Date exp = new Date(now.getTime() + 1000L * 60 * 60 * 24 * 14); // 14일  
  
    return Jwts.builder()  
        .setClaims(claims)  
        .setIssuedAt(now)  
        .setExpiration(exp)  
        .signWith(REFRESH_KEY, SignatureAlgorithm.HS512)  
        .compact();  
}
```

6. 보안과 인증

실습(JWT)

```
public Claims parseAccessClaims(String token) {  
    return Jwts.parserBuilder()  
        .setSigningKey(ACCESS_KEY)  
        .build()  
        .parseClaimsJws(token)  
        .getBody();  
}  
  
public Claims parseRefreshClaims(String token) {  
    return Jwts.parserBuilder()  
        .setSigningKey(REFRESH_KEY)  
        .build()  
        .parseClaimsJws(token)  
        .getBody();  
}  
  
public String extractUsername(String token) {  
    return parseAccessClaims(token).getSubject();  
}
```

```
/** Access 토큰 만료 여부 */  
public boolean isAccessExpired(String accessToken) {  
    return parseAccessClaims(accessToken).getExpiration().before(new  
Date());  
}  
  
/** Refresh 토큰 만료 여부 */  
public boolean isRefreshExpired(String refreshToken) {  
    return parseRefreshClaims(refreshToken).getExpiration().before(new  
Date());  
}  
}
```

6. 보안과 인증

실습(JWT)

```
// JwtFilter.java
package com.example.demo.security;

@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        if (SecurityContextHolder.getContext().getAuthentication() != null) {
            filterChain.doFilter(request, response);
            return;
        }
    }
}
```

```
String header = request.getHeader("Authorization");
if (header != null && header.startsWith("Bearer ")) {
    String token = header.substring(7);
    if (token.isBlank()) {
        filterChain.doFilter(request, response);
        return;
    }

    try {
        Claims claims = jwtUtil.parseAccessClaims(token);
        String type = String.valueOf(claims.get("type"));
        if (!"access".equals(type)) {
            SecurityContextHolder.clearContext();
        } else {
            String username = claims.getSubject();
            UserDetails user =
                userDetailsService.loadUserByUsername(username);
            UsernamePasswordAuthenticationToken auth =
                new UsernamePasswordAuthenticationToken(user, null,
                user.getAuthorities());
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
    } catch (io.jsonwebtoken.ExpiredJwtException e) {
        SecurityContextHolder.clearContext();
    } catch (Exception e) {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        return;
    }
    filterChain.doFilter(request, response);
}
}
```

6. 보안과 인증

실습(JWT)

```
// MyUserDetailsService.java
```

```
package com.example.demo.security;
```

```
import com.example.demo.entity.Member;
```

```
import com.example.demo.repository.MemberRepository;
```

```
import lombok.RequiredArgsConstructor;
```

```
import org.springframework.security.core.userdetails.*;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
@RequiredArgsConstructor
```

```
public class MyUserDetailsService implements UserDetailsService {
```

```
    private final MemberRepository memberRepository;
```

```
    @Override
```

```
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
```

```
        Member m = memberRepository.findByEmail(email)
```

```
            .orElseThrow(() -> new UsernameNotFoundException("not found"));
```

```
        return new CustomUserDetails(
```

```
            m.getEmail(),
```

```
            m.getPassword(),
```

```
            m.getRole().name());
```

```
    }
```

```
}
```

6. 보안과 인증

실습(JWT)

```
// MemberRepository.java
package com.example.demo.repository;

import com.example.demo.entity.Member;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface MemberRepository extends JpaRepository<Member, Long> {
    Optional<Member> findByEmail(String email);
}
```

6. 보안과 인증

실습(JWT)

// Member.java

```
package com.example.demo.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.*;
```

```
@Entity
```

```
@Table(name = "members")
```

```
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
```

```
public class Member {
```

```
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @Column(unique = true, nullable = false)
```

```
    private String username;
```

```
    @Column(nullable = false)
```

```
    private String password;
```

```
    @Enumerated(EnumType.STRING)
```

```
    @Builder.Default
```

```
    private Role role = Role.USER;
```

```
}
```

// Role.java

```
package com.example.demo.entity;
```

```
public enum Role {
```

```
    ADMIN, USER
```

```
}
```

6. 보안과 인증

실습(JWT)

// LoginDTO.java

```
package com.example.demo.dto;
```

```
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;
```

```
@Getter @Setter @NoArgsConstructor @ToString
public class LoginDTO {
```

```
    private String email;
    private String password;
}
```

// TokenRefreshDTO.java

```
package com.example.demo.dto;
```

```
import lombok.Data;
```

```
@Data
public class TokenRefreshDTO {
    private String refreshToken;
}
```

// MemberDTO.java

```
package com.example.demo.dto;
```

```
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;
```

```
@Getter @Setter @NoArgsConstructor @ToString
public class MemberDTO {
```

```
    private Long id;
    private String username;
    private String email;
    private String password;
```

```
    @Builder.Default
    private Role role = Role.USER;
}
```

6. 보안과 인증

실습(JWT)

// AuthController.java

```
package com.example.demo.controller;
```

```
@RestController
```

```
@RequestMapping("/auth")
```

```
@RequiredArgsConstructor
```

```
public class AuthController {
```

```
    private final MemberService memberService;
```

```
    private final AuthenticationManager authenticationManager;
```

```
    private final JwtUtil jwtUtil;
```

```
    private final RedisTemplate<String, String> redisTemplate;
```

```
    @PostMapping("/signup")
```

```
    public void signup(@RequestBody MemberDTO dto) {
```

```
        memberService.save(dto);
```

```
    }
```

```
    @PostMapping("/login")
```

```
    public ResponseEntity<?> login(@RequestBody LoginDTO dto, HttpServletResponse res) {
```

```
        authenticationManager.authenticate(
```

```
            new UsernamePasswordAuthenticationToken(dto.getEmail(), dto.getPassword())
```

```
        );
```

```
        MemberDTO memberDTO = memberService.login(dto);
```

```
        String accessToken = jwtUtil.createToken(dto.getEmail(), memberDTO.getRole().name());
```

```
        String refreshToken = jwtUtil.createRefreshToken(dto.getEmail(),  
memberDTO.getRole().name());
```

```
        redisTemplate.opsForValue().set(dto.getEmail(), refreshToken, 20, TimeUnit.DAYS);
```

```
        Map<String, Object> loginInfo = new HashMap<>();
```

```
        loginInfo.put("id", memberDTO.getId());
```

```
        loginInfo.put("token", accessToken);
```

```
        loginInfo.put("refreshToken", refreshToken);
```

```
        return new ResponseEntity<>(loginInfo, HttpStatus.OK);
```

```
    }
```

6. 보안과 인증

실습(JWT)

```

@PostMapping("/refresh")
public ResponseEntity<?> generateNewRt(@RequestBody TokenRefreshDTO dto ) {

    if (dto == null || dto.getRefreshToken() == null || dto.getRefreshToken().isBlank()) {
        return new ResponseEntity<>("refreshToken is required", HttpStatus.BAD_REQUEST);
    }
    Claims claims;
    try {
        claims = jwtUtil.parseRefreshClaims(dto.getRefreshToken());
        Object typeObj = claims.get("type");
        if (!(typeObj instanceof String) || !"refresh".equals(typeObj)) {
            return new ResponseEntity<>("invalid token type", HttpStatus.UNAUTHORIZED);
        }
    } catch (JwtException e) {
        return new ResponseEntity<>("invalid refresh token", HttpStatus.UNAUTHORIZED);
    }

    String email = claims.getSubject();
    Object rt = redisTemplate.opsForValue().get(email);
    if (rt == null || !rt.toString().equals(dto.getRefreshToken())) {
        return new ResponseEntity<>("refresh token mismatch", HttpStatus.UNAUTHORIZED);
    }
    String role = String.valueOf(claims.get("role"));
    String newAccess = jwtUtil.createToken(email /* or memberId */, role);
    Map<String, Object> body = new HashMap<>();
    body.put("token", newAccess);
    return new ResponseEntity<>(body, HttpStatus.OK);
}

```

6. 보안과 인증

실습(JWT)

```
@PostMapping("/logout")
public ResponseEntity<?> logout(@RequestBody TokenRefreshDTO dto) {

    if (dto == null || dto.getRefreshToken() == null || dto.getRefreshToken().isBlank()) {
        return new ResponseEntity<>("refreshToken is required", HttpStatus.BAD_REQUEST);
    }
    try {

        Claims claims = jwtUtil.parseRefreshClaims(dto.getRefreshToken());
        String subject = claims.getSubject();
        redisTemplate.delete(subject);

    } catch (JwtException e) {
        return new ResponseEntity<>("invalid refresh token", HttpStatus.UNAUTHORIZED);
    }

    return new ResponseEntity<>(Map.of("message", "logged out"), HttpStatus.OK);
}
```

6. 보안과 인증

실습(JWT)

```
// MemberController.java
package com.example.demo.controller;

import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class MemberController {

    // 마이페이지: 인증 필요
    @GetMapping("/me")
    public String me(@AuthenticationPrincipal UserDetails user) {

        return "Hello, " + user.getUsername(); // email
    }
}
```

1.5 보안과 인증

실습(JWT) – 회원가입

POST http://localhost:8090/auth/signup

Headers: Content-Type:application/json

BODY: { "username": "park", "email": "park@test.com", "password": "P@ssw0rd123!", "role": "USER" }

08_Basic1_JWT_signup



Save

METHOD

SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]

POST

http://localhost:8090/auth/signup

Send

length: 33 byte(s)

▶ QUERY PARAMETERS

HEADERS ⑦ ↓

Form

BODY ⑦

Text

☒ Content-T : application/json

+ Add header

⌕ Add authorization



```
1 { "username": "park",
2   "email": "park@test.com",
3   "password": "P@ssw0rd123!",
4   "role": "USER"
5 }
```

Result Grid					
Filter Rows:					
Edit:					
Export/Import:					
	id	email	password	username	role
▶	1	park@test.com	\$2a\$10\$ORpnoI68NMOfa3oi.7qzdO9asSvqK3Sr...	park	USER
*	NULL	NULL	NULL	NULL	NULL

6. 보안과 인증

실습(JWT) – 로그인

POST http://localhost:8090/auth/login

Headers: Content-Type:application/json

BODY: {"email":"park@test.com", "password": "P@ssw0rd123!" }

Response

Cache Detected - Elapsed Time: 383ms

200

HEADERS ⑦

pretty ▼

Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
X-Content-Type-Option... nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0

BODY ⑦

pretty ▼

```
{  
  id: 1,  
  token: "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJwYXJrQHRlc3QuY29tIiwicm9sZSI6IiVTRVIlLCJ0eXB1IjoieWNjZ  
  refreshToken: "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJwYXJrQHRlc3QuY29tIiwicm9sZSI6IiVTRVIlLCJ0eXB1Ij
```

lines nums [copy](#)

length: 487 bytes

6. 보안과 인증

실습(JWT) – me (Access Token 유효)

GET http://localhost:8090/api/me

Headers: Authorization: Bearer <access token>

08_Basic3_JWT_me Save

METHOD GET SCHEME // HOST [":" PORT] [PATH ["?" QUERY]] Send
http://localhost:8090/api/me length: 28 byte(s)

QUERY PARAMETERS

HEADERS 1/2 Form BODY 1

☒ Authorization : Bearer eyJhbGciOiJIUzUxMiJ9.eyJzdWUiOiJwYXk@test.comJwYy> X + +

+ Add header + Add authorization

Response Cache Detected - Elapsed Time: 20ms

200

HEADERS 1 pretty BODY raw

Vary: Origin
Vary: Access-Control-Request-Method

Hello, park@test.com

6. 보안과 인증

실습(JWT) – me (Access Token 만료 후)

GET http://localhost:8090/api/me

Headers: Authorization: Bearer <access token>

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: GET
 - URL: http://localhost:8090/api/me
 - Headers: Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJwYyY
- Response:**
 - Status: 401 (Unauthorized)
 - Body: { "status": 401, "error": "UNAUTHORIZED", "message": "인증이 필요합니다." }

The 401 status and the response body are highlighted with red boxes in the original image.

POST http://localhost:8090/auth/refresh

Headers: Content-Type:application/json

BODY: {"refreshToken":"refres토큰" }

Response

200

A screenshot of a web browser's developer tools interface. The left pane shows the 'HEADERS' tab with a list of response headers: 'Vary: Origin', 'Vary: Access-Control-Request-Method', 'Vary: Access-Control-Request-Headers', and 'X-Content-Type-Options: nosniff'. The right pane shows the 'BODY' tab with a JSON object containing a single key-value pair: 'token: "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJwYXJrQHRLc...''. A red rectangular box highlights the entire JSON object in the BODY pane.

6. 보안과 인증

실습(JWT) – logout

POST http://localhost:8090/auth/logout

Headers: Content-Type:application/json

BODY: {"refreshToken":"refres토큰" }

08_Basic5_JWT_logout Save

METHOD POST SCHEME // HOST [":" PORT] [PATH ["?" QUERY]] Send

length: 33 byte(s)

▶ QUERY PARAMETERS

HEADERS 1/2 Form

☒ Content-Type : application/json ×

+ Add header ⌵ Add authorization 🗑

BODY Text

```
{
  "refreshToken": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJwYXJlbnQ"
}
```

Response

200

HEADERS ? pretty

Vary: Origin

Vary: Access-Control-Request-Method

Vary: Access-Control-Request-Headers

X-Content-Type-Options: nosniff

BODY ?

```
{
  "message": "logged out"
}
```

6. 정리

요약맵

- REST 개요 및 특징
- REST 설계 원칙
- @RestController 개요
- REST API 테스트 도구
- 요청/응답 처리 심화 (ApiResponse, ReponseEntity, @RestControllerAdvice)
- REST API 문서화 개요
- Swagger 및 OpenAPI
- Spring Security 개요 및 아키텍처
- JWT 개요
- Refresh Token

수고하셨습니다.